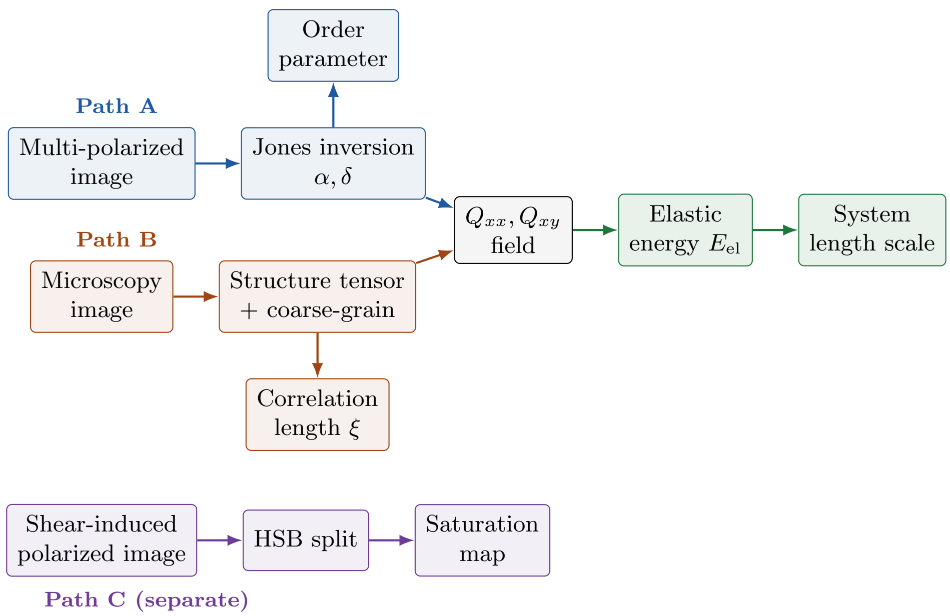
From Microscopy Image to Nematic Elastic Energy

This report documents a new branch of BARCODE. It extends the existing workflow so that a raw microscopy image is turned into a 2D nematic Q-tensor field (Q_{xx}, Q_{xy}), which is then fed into the Landau–de Gennes elastic-energy density. Two independent front-ends produce the Q-tensor — **Path A** (multi-polarized imaging) and **Path B** (microscopy fluorescence + structure-tensor edge detection) — and either one feeds the same generic energy stage. A third input, **Path C** (shear-induced polarized imaging), is a *separate readout*: it does not produce a Q-tensor and does not feed the energy stage; it yields a saturation map via an HSB channel split.



The two Q-producing paths differ in what grid Q lives on: Path A returns a *per-pixel* Q read directly from the optics; Path B returns a *coarse-grained* Q on a window grid inferred from image gradients. The energy stage is agnostic — it takes any Q_{ij} array plus its grid spacing (dx, dy). Path C runs alongside as an independent saturation readout.

Path A — Multi-polarized image → Q-tensor

Reconstruct Q from a raw multi-polarized image (Sokolov et al., *Adv. Mater.* **37**, 2418846, 2025). The sensor interleaves four micro-polarizer angles in a 2×2 super-pixel; splitting the mosaic gives $I_0, I_{45}, I_{90}, I_{135}$, and a Jones inversion turns them into director angle α and retardance δ:

$$\boxed{A = \frac{I_0 - I_{90}}{\mathrm{tot}}, \quad B = \frac{I_{45} - I_{135}}{\mathrm{tot}}, \qquad \delta = \arcsin\sqrt{A^2 + B^2}, \quad \alpha = \tfrac{1}{2}\,\mathrm{atan2}(B, A)}$$

```python
def reconstruct(channels):
    I0, I45, I90, I135 = channels
    tot = (I0+I45+I90+I135)/2 + 1e-12
    A, B  = (I0-I90)/tot, (I45-I135)/tot
    delta = np.arcsin(np.clip(np.sqrt(A**2+B**2), 0, 1))    # retardance
    alpha = 0.5*np.arctan2(B, A)                            # director angle
    return alpha, delta
```

The Q-tensor is built from (α, δ) using retardance as the amplitude:

$$Q_{xx} = \tfrac{\delta}{2}\cos 2\alpha, \qquad Q_{xy} = \tfrac{\delta}{2}\sin 2\alpha$$

The scalar order parameter is $\langle\sqrt{A^2 + B^2}\rangle$; besides feeding the Q-tensor, it is exported as a stand-alone side output of this path.

Path B — Microscopy image → Q-tensor

Here the director is inferred from image *gradients*: edges of rod-shaped bacteria set a local gradient direction, and the alignment axis is its perpendicular. After preprocessing (grayscale → illumination correction → foreground mask), Sobel gradients form the smoothed *structure tensor*, window-averaged and diagonalized:

$$J = G_\sigma * \begin{pmatrix} G_x^2 & G_x G_y \\ G_x G_y & G_y^2 \end{pmatrix}, \qquad \theta = \tfrac{1}{2}\,\mathrm{atan2}(2\bar{J}_{xy},\ \bar{J}_{xx} - \bar{J}_{yy}) + \tfrac{\pi}{2}$$

$$S = \frac{\sqrt{(\bar{J}_{xx} - \bar{J}_{yy})^2 + 4\bar{J}_{xy}^2}}{\bar{J}_{xx} + \bar{J}_{yy}}, \qquad Q_{xx} = \tfrac{1}{2}S\cos 2\theta, \quad Q_{xy} = \tfrac{1}{2}S\sin 2\theta$$

```
Gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)      # edge detection
Gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
Jxx, Jyy, Jxy = smooth(Gx*Gx), smooth(Gy*Gy), smooth(Gx*Gy)
# ... window-average J over foreground, then:
theta_dir = 0.5*np.arctan2(2*jxy, jxx - jyy) + np.pi/2   # director _|_ gradient
coherence = np.sqrt((jxx-jyy)**2 + 4*jxy**2) / (jxx+jyy)
Qxx = 0.5*coherence*np.cos(2*theta_dir)
Qxy = 0.5*coherence*np.sin(2*theta_dir)
```

The same code also measures the nematic correlation length ξ by fitting $C(r) = A\,e^{-r/\xi}$ to the FFT autocorrelation of Q-fluctuations. This ξ is exported as a stand-alone side output of Path B (it is not required by the energy stage).

Path C — Shear-induced polarized image → saturation (separate readout)

This input is handled differently from Paths A and B: it does *not* produce a Q-tensor and does *not* feed the elastic-energy stage. A shear-induced polarized image is converted from RGB to the HSB (hue–saturation–brightness) colour space, and the **saturation** channel is kept as the output map. In a shear-birefringence image, colour *hue* tracks the local optical retardance/orientation while *saturation* reflects how strongly polarized (how birefringent) each region is — so the saturation map serves as a direct, per-pixel readout of shear-induced alignment strength, without any tensor reconstruction.

$$(H, S, B) = \mathrm{rgb\_to\_hsb}(\mathrm{image}), \qquad \mathrm{output} = S$$

```
import colorsys, numpy as np
def saturation_map(rgb):
    rgb = rgb[..., :3].astype(float) / 255.0          # drop alpha, normalize
    r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]
    mx = np.max(rgb, axis=-1); mn = np.min(rgb, axis=-1)
    S = np.where(mx > 0, (mx - mn) / mx, 0.0)          # HSB saturation
    return S                                           # H (orientation) and B (intensity) kept aside
```

The hue (H) and brightness (B) channels are computed by the same split but not used for the alignment readout here: H encodes orientation and B overall transmitted intensity, whereas S isolates the birefringence strength this branch is after.

Bridge — both Q-paths give the same object

Both front-ends return the two independent components of a traceless symmetric 2D Q-tensor, stored as float32 (`Qxx`, `Qxy`); the third is implied:

$$\boxed{Q = \begin{pmatrix} Q_{xx} & Q_{xy} \\ Q_{xy} & -Q_{xx} \end{pmatrix}, \qquad Q_{yy} = -Q_{xx}}$$

The only thing the energy stage needs beyond (Q_{xx}, Q_{xy}) is the grid spacing (dx, dy): per-pixel spacing for Path A, the window step for Path B. The energy code reconstructs the full 2×2 $Q[i][j]$ from these two components before differentiating.

Stage 3 — Q-tensor → Elastic Energy

Feed the field into the 2D Landau–de Gennes elastic energy density with constants L_1, L_2, L_6, as given by Eq. (2) of Emersic, de Pablo, Snezhko & Sokolov, *Phys. Rev. Lett.* **135**, 048301 (2025). The routine builds every derivative $\partial_k Q_{ij}$ and accumulates the three elastic terms defined there:

```
def elastic_energy_2d(Q, dx, dy, L1, L2, L6):
    dQ = [[grad(Q[i][j], dx, dy) for j in range(2)] for i in range(2)]  # dQ_ij/dx_k
    T1 = T2 = T3 = np.zeros(Q[0][0].shape)
    for i in range(2):
      for j in range(2):
        for k in range(2):
          T1 += dQ[i][j][k]**2                      # (dQ_ij/dx_k)^2
          T2 += dQ[i][j][j] * dQ[i][k][k]           # (dQ_ij/dx_j)(dQ_ik/dx_k)
          for l in range(2):
            T3 += Q[l][k] * dQ[i][j][l] * dQ[i][j][k]
    return 0.5*L1*T1 + 0.5*L2*T2 + 0.5*L6*T3, ...
```

Elastic constants. Frank constants (splay K_{11}, bend K_{33}) map to the L_i; in 2D only $(2L_1 + L_2)$ and L_6 are fixed, so $L_1 = 0$ and L_2 carries the isotropic part:

$$\boxed{L_2 = \frac{K_{11} + K_{33}}{2S^2} \text{ (trace 2)}, \quad L_2 = \frac{2K_{11} + K_{33}}{3S^2} \text{ (trace 3)}, \quad L_6 = \frac{K_{33} - K_{11}}{2S^3}}$$

Validation

Evaluated on an annulus for four analytic textures (pure splay, pure bend, $\pm\frac{1}{2}$ defects) at $K_{11} = 0.4,\ K_{33} = 1.0,\ S = 0.5$:

```
trace=2: L1=0.0000 L2=2.8000 L6=2.4000 -> K=(0.4, 1.0)
  pure splay / bend / +1/2 / -1/2: max rel err 1.8e-03 -- 4.3e-03
uniform director:  max |E| = 1.26e-29
paper's L_i = (0.4, 1.6, 2.4) -> read back trace=3:  (0.4, 1.0), trace=2:  (0.3, 0.9)
```

All four textures reproduce their exact Frank energies to $\sim 10^{-3}$ relative error, a uniform director gives machine-zero energy, and the paper's L_i recover $(0.4, 1.0)$ only under the trace $= 3$ convention — so the energy stage must be handed a trace $= 3$ Q-tensor (or the constants re-derived) to match the paper.

Stage 4 — Elastic Energy → System Length Scale

The elastic-energy density defines a characteristic *system length scale* ℓ by balancing the elastic term against a reference energy scale of the system. Writing the mean elastic energy density as $\bar{E}_{\mathrm{el}}$

and using a representative elastic constant K (from the L_i via `K_from_L`), the length scale follows from dimensional balance $K/\ell^2 \sim \bar{E}_{\text{el}}$:

$$\boxed{\ell = \sqrt{\frac{K}{\bar{E}_{\text{el}}}}}$$

This turns the energy-density map into a single physical length characterizing the scale over which elastic distortions are stored — the final output of the pipeline.

End-to-end: image $\rightarrow$ [Path A multi-polarized *or* Path B microscopy] $\rightarrow$ (Q_{xx}, Q_{xy}) field $\rightarrow$ `build_Q` (to full 2×2) $\rightarrow$ `elastic_energy_2d`$(Q, dx, dy, L_1, L_2, L_6)$ $\rightarrow$ energy-density map $\rightarrow$ system length scale ℓ. *Side outputs:* Path A also yields the order parameter, Path B the correlation length ξ.